



THE SCHOOL OF AI



NAVTTC

Variables and f-strings

Day 2 — Storing things, naming them well, and building text out of them. By the end of today you can write a prompt template that fills itself in.

Four hours, one payoff

Everything today builds to the last hour, where you write text that assembles itself from data.

1

Hour 1 · Variables

What they really are, how to name them so your code stays readable, and the four types you will actually use.

2

Hour 2 · Strings

The handful of string methods that do ninety per cent of real work. Your messy lists get cleaned properly.

3

Hour 3 · f-strings

Building text from variables, formatting numbers, and the trap everyone falls into once.

4

Hour 4 · The payoff

A prompt template that fills itself in — the bridge between this track and your main course.

A prompt is just text. Text you build is a template.

In your main course you write one prompt at a time, by hand. After today you can write one prompt shape and have it fill itself in forty times with different data.

A labelled box, nothing more

Four facts. Get these right and most beginner confusion disappears.

1 It stores one thing

A name, a number, a list. The box holds whatever you put in it, and the label is how you get it back.

2 It reads right to left

`total = 500` means: take 500, put it in the box called total. Not "total equals 500" as in mathematics.

3 It can change

Assign to it again and the old value is gone. That is normal, and it is how a counter or a running total works.

4 The name is for humans

Python does not care whether you write `x` or `monthly_total`. The person reading it in March does.

Names are the cheapest documentation

DO NOT WRITE	WRITE	WHY
x, a1, data2	order_total, client_name	You will read this again in a month and remember nothing
clientName	client_name	Python's convention is lower case with underscores
Print, List	print, list	Case matters. Capitals are for something else entirely
total = ... twice	total_before, total_after	Reusing one name for two meanings is how bugs hide
I, O, l	Anything else	They look like the digits one and zero in most fonts

An AI reads your variable names too. Good names get you better suggestions.

Four types will carry you a long way

There are more. These four cover almost everything you will write this year.



str — text

Anything in quotes.
"Lahore", "03001234567". A
phone number is text, not a
number.



int — whole number

42, 0, -7. Counts of things:
orders, rows, days. No
decimal point.



float — decimal

1250.75, 0.15. Money,
percentages,
measurements. Careful with
exact comparisons.



bool — true or false

True or False, capitalised.
Answers a yes-or-no
question. Powers every if
statement.

The error you will meet today

TYPE THIS AND READ THE ERROR BEFORE YOU FIX IT

```
orders = "12"          # text, because of the quotes
print(orders + 3)      # TypeError

print(int(orders) + 3) # 15 – converted first
print(type(orders))   # <class 'str'>
```

Every number that arrives from a file, a form or a website is text.

Converting it before you calculate is one of the most common lines in real data work.

Variables and types

New notebook, typed by hand. Everything goes in your task file with the output written down.

1 Describe one order

Five variables about a real or invented order: client name, city, item, quantity, unit price. Name them properly.

2 Calculate

Work out the total from quantity and unit price. Print it. Then change the quantity and run it again.

3 Cause a TypeError

Put the price in quotes and try the calculation. Read the error, then fix it with a conversion.

4 Check your types

Print type() of all five variables. Is the phone number or product code text, as it should be?

Seven methods do most of the work

METHOD	WHAT IT DOES	WHEN YOU REACH FOR IT
<code>.strip()</code>	Removes spaces from both ends	Every single time data comes from a human
<code>.title()</code> <code>.upper()</code> <code>.lower()</code>	Changes capitalisation	Tidying names, comparing things fairly
<code>.replace(a, b)</code>	Swaps one piece of text for another	Fixing a recurring typo or stripping a symbol
<code>.split(",")</code>	Cuts text into a list at each separator	Turning one messy line into separate fields
<code>", ".join(items)</code>	Glues a list back into one string	Building a readable line out of a list
<code>len(text)</code>	How many characters	Checking a message fits a limit

These chain: `raw.strip().lower().replace(" ", " ")` runs left to right.

Your messy list, cleaned

We use a real list from someone in this room. Type along with your own.

ONE MESSY LINE IN, A CLEAN LIST OUT

```
raw = "ali khan, SARA AHMED , bilal ,Ayesha Khan"

parts = raw.split(",")

for p in parts:
    clean = p.strip().title().replace(" ", " ")
    print(clean)
```

Put the variable inside the sentence

The letter f before the quotes, and curly braces around anything you want filled in.

THE F IS NOT OPTIONAL

```
client = "Sara Ahmed"  
items = 3  
total = 4250.5  
  
print(f"Salam {client}, your {items} items come to Rs {total}.")  
  
# Salam Sara Ahmed, your 3 items come to Rs 4250.5.
```

Forget the f and Python prints the braces exactly as you typed them.

Making it look like an invoice

A COLON INSIDE THE BRACES CONTROLS THE SHAPE

```
total = 4250.5
rate = 0.17

print(f"Rs {total:,.2f}")          # Rs 4,250.50
print(f"Tax: {rate:.0%}")          # Tax: 17%
print(f"{'Item':<12}{'Price':>8}") # columns that line up
```

This is the difference between output and a document.

A client sees Rs 4,250.50 and thinks you are careful. They see 4250.5 and wonder what else you skipped.

You may see t-strings

Python 3.14 added template strings, written with a t instead of an f. They solve a problem you do not have yet.

1	What they are	A t-string does not produce finished text. It produces an object that another piece of code processes safely before display.
2	What they are for	Building HTML, SQL or shell commands from untrusted input, where blindly inserting text is a security hole.
3	Why not today	You are printing to a screen and building prompts. f-strings are the right tool, and they are not going anywhere.
4	Check what you have	<code>import sys; print(sys.version)</code> tells you which Python your notebook is running. Tutorials assume, you verify.

A prompt template that fills itself

This is a Week 4 prompt from AI for Everyone — with the variable parts left open.

TRIPLE QUOTES HOLD A PROMPT THAT RUNS OVER SEVERAL LINES

```
role = "an export sales manager"
client = "Sara Ahmed"
product = "cotton bed linen"

prompt = f"""You are {role} writing to {client}.
Write a short follow-up about our {product} quote.
Polite, direct, under 120 words, British English."""

print(prompt)
```

Build a message generator

For the business you invented in the main course, or your own. Typed by hand, output written down.

1	Set your variables	Client name, city, product, quantity, unit price, delivery day. Named properly, with the right types.
2	Calculate the total	Quantity times price. Then format it as money with a comma and two decimals.
3	Write the message	One f-string, several lines, using every variable. It should read like something you would actually send.
4	Change one variable	A different client and quantity. Run it again. Nothing else in your code should need touching.
5	Now make it a prompt	Rewrite the same message as a prompt for an AI tool, with the variable parts filled in the same way.

Four mistakes everyone makes once

Three of these produce no error message at all, which is what makes them worth memorising.



Forgot the f

Prints the braces literally.
No error. Check the first character before the quote.



Nested quotes

Use single inside double, or the string ends early and everything after it breaks.



Number kept as text

"3" * 2 gives 33. It runs, and the answer is wrong. Print the type when in doubt.



Reused a variable

total set twice for two different things. The first value is gone and you will not be told.

Twenty minutes, again

Same rule: type it yourself first, ask the assistant second, retype anything you accept.

1 Rebuild the generator

From scratch in a new notebook, without the slides. Note exactly where you had to stop and think.

2 Clean your own list

The messy list you brought today. Split it, strip it, fix the capitals, print it clean.

3 Write three formats

One money amount, one percentage, and two columns that line up. Screenshot the output.

Bring a small CSV or spreadsheet next time.

END OF DAY 2

You can now write text that writes itself.

Next · Day 3 — lists and loops properly, reading a real file, and turning today's single message into forty of them without touching the code twice.